

Laboratory of Mobile Operating Systems

Android Intents

Introduction

Intents are special messages used for communicating with various components of the Android operating system. Intents are used for tasks such as sending data, binding to services and running applications. You will need an Intent even if you want to start a different activity inside your own app:

```
val secondActivityResult =  
    Intent(this, SecondActivity::class.java)  
    startActivity(secondActivityResult)
```

Before starting the second activity, you can also prepare extra data to be sent, like a Boolean:

```
secondActivityResult.putExtra("SomeValue", true)
```

Inside your second activity, this extra data can be read at any time:

```
val receivedData = intent.getBooleanExtra("SomeValue", false)
```

The second activity can also set a return value:

```
val data = Intent()  
data.putExtra("IntResult", 5)  
setResult(Activity.RESULT_OK, data)
```

Unfortunately, obtaining this result in the main activity is not so straightforward. First, you should prepare a special launcher along with some logic for reading the result:

```
val startForResultActivity = registerForActivityResult(  
    ActivityResultContracts.StartActivityForResult()) {  
    result: ActivityResult ->  
        if (result.resultCode == Activity.RESULT_OK) {  
            val data: Intent? = result.data  
            val returnedValue = data?.getIntExtra("IntResult", -1)  
        }  
    }
```

Finally, you will need to replace `startActivity(secondActivityResult)` with the following code:

```
startForResultActivity.launch(secondActivityResult)
```

Task 0.

Launch Android Studio and create a new project based on the Empty Views Activity template. Alternatively, you can also use your previous project if you prefer to do so.

Task 1. (1 point)

Add a new button to your UI layout and a second activity to your project. When the button is clicked, the second activity should be started.

Task 2. (1 points)

Create a new class in your project, used for generating a simple image. Use it to draw at least two different shapes with at least two different colors. Feel free to start with the following code: use *canvas* methods for drawing shapes and *paint* methods for adjusting colors and styles. Note: you won't be able to see your results before completing the next task.

```
class CustomView : View {
    constructor(context: Context) : super(context)
    constructor(context: Context, attrs: AttributeSet) :
super(context, attrs)
    constructor(context: Context, attrs: AttributeSet, defStyle:
Int) : super(context, attrs, defStyle)

    override fun draw(canvas: Canvas) {
        super.draw(canvas)
        val paint = Paint()
        // Paint here.
    }
}
```

Task 3. (2 points)

Place your custom view inside the layout of one of your activities, then add a Switch component to your other activity. When the Switch component is checked, your custom view should use different colors or draw different shapes. You may also use a different component than Switch, such as Checkbox or ToggleButton, as long as it can hold a value.

Variant I

The Switch goes to the main activity and the custom view goes to the second activity.

Variant II

The custom view goes to the main activity and the Switch goes to the second activity.

If you have trouble placing your custom view in *Design* mode, try using the *Code* mode:

```
<view
    android:id="@+id/customView"
    class="ksg.mso.intents.CustomView"
    android:layout_width="0dp"
    android:layout_height="0dp"
    app:layout_constraintBottom_toBottomOf="parent"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toTopOf="parent" />
```

Task 4. (1 point)

Variant I

Add a new button and an `ImageView` to your main activity. Pressing the button should start the system camera app. After a picture is taken, the `ImageView` should display its thumbnail. You can construct the intent like this:

```
val intent = Intent(MediaStore.ACTION_IMAGE_CAPTURE)
```

After obtaining the result returned by the camera, you can get the thumbnail like this:

```
val data: Intent? = result.data
data?.let {
    val thumbnail: Bitmap? = IntentCompat
        .getParcelableExtra(data, "data", Bitmap::class.java)
    thumbnail?.let {
        captureImageView.setImageBitmap(thumbnail)
    }
}
```

Variant II

Add a new button to your main activity. Pressing this button should call the system alarm clock. Use one of the following:

```
val intent = Intent(AlarmClock.ACTION_SET_TIMER)
```

or:

```
val intent = Intent(AlarmClock.ACTION_SET_ALARM )
```

You will also need to update you *AndroidManifest.xml* file by adding the following line before or after the `<application>` node:

```
<uses-permission
    android:name="com.android.alarm.permission.SET_ALARM" />
```